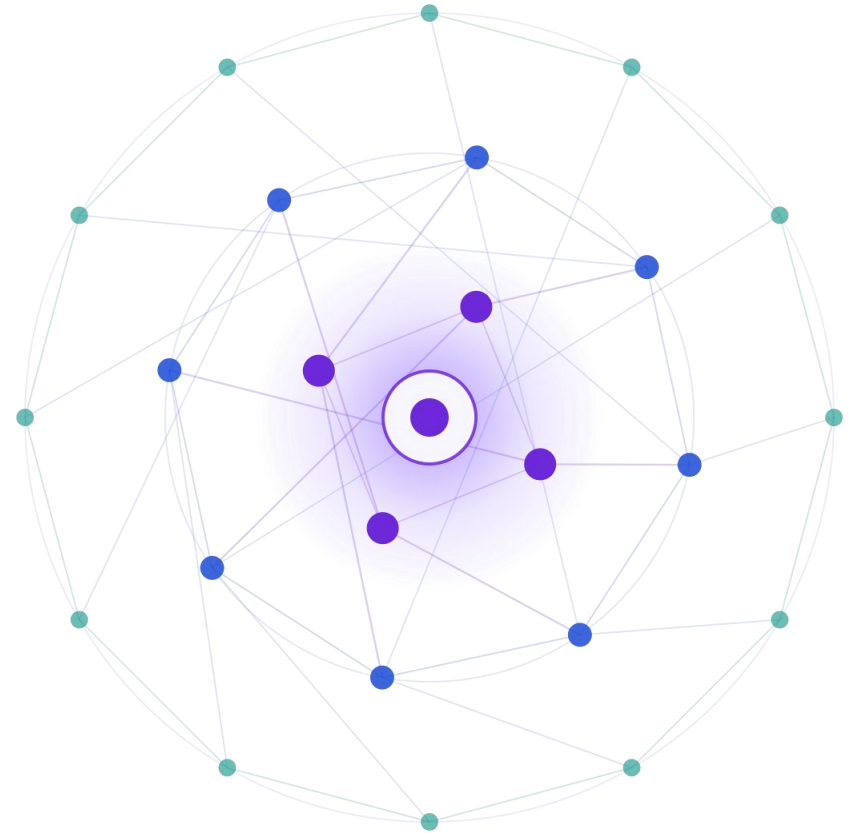


Generative AI Hackathon

3rd Edition 2026 – 27

- Project - **Axiom: multi-pass agentic code retrieval**
- Theme ID - **01** • **Agentic Code Intelligence**
- Team Name - **Incognito**
- College Name - **Thapar Institute of Engineering & Technology, Patiala**
- Member Name & Email 1- **Prabinder Singh** · psingh16_be24@thapar.edu
- Member Name & Email 2- **Anish Grover** · agrover_be24@thapar.edu
- Member Name & Email 3- **Harshdeep Athawale** · hathawale_be24@thapar.edu
- Member Name & Email 4- **Parth Deshmukh** · pdeshmukh_be24@thapar.edu
- Submission Github link - github.com/HarshdeepAthawale/AXIOM



Theme

THEME 01 · AGENTIC CODE INTELLIGENCE

Engineers rarely ask for new code.
They ask: **where does this already happen?**

Voice-assistant codebases span thousands of JavaScript files and dozens of agents and tools. Given a natural-language query, Axiom returns a ranked list of code snippets with exact file:line locations.

Retrieval, not generation: we find and rank existing code, we never write it.

Too large for any LLM context window

CPU-only

Changes every commit

Goals: P0 retrieval accuracy (CoIR AppsRetrieval) · P1 retrieval across versions · Bonus: evolutionary retrieval over all versions at once

Q1

SEMANTIC

How is the input preprocessed before going to the main function?

“preprocess” appears nowhere in the code; normalize() does

Q2

STRUCTURAL

Which files call tool XYZ before tool ABC?

A question about call order, which is not in any snippet’s text

Q3

USAGE

Where is the Bluetooth-settings deeplink used?

An exact literal string that semantic search dilutes

Existing Solutions & Gaps



Keyword search

grep, IDE find, plain BM25

GOOD AT

Exact identifiers, literal strings, fast on CPU

GAPS

- Misses meaning: “preprocessing” never matches normalize()
- Cannot express “calls X before Y”



Embedding RAG

chunk, embed, nearest neighbours

GOOD AT

Semantic similarity across vocabulary gaps

GAPS

- Dilutes exact literals like a deeplink string
- Blind to call graphs and execution order
- Re-embeds everything when code changes



Long-context LLM

paste the repository into a prompt

GOOD AT

Fluent explanations over small code

GAPS

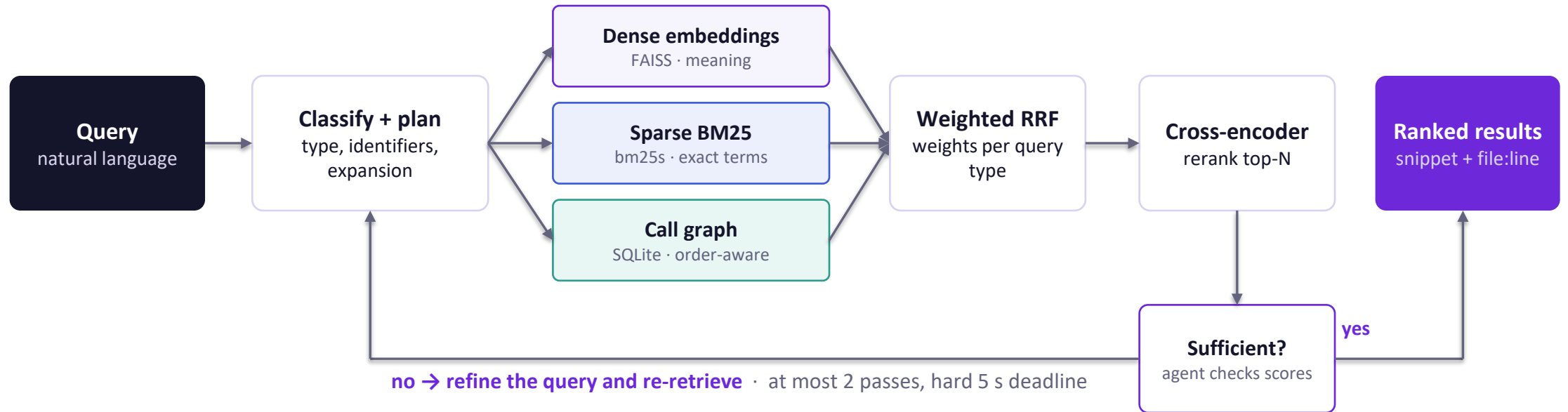
- The codebase does not fit in any context window
- Needs a GPU or a cloud API; code leaves the machine
- Returns prose, not ranked file:line locations

The gap: no single signal answers all three query types, none preserve call order, and none keep an index fresh across versions without re-embedding everything.

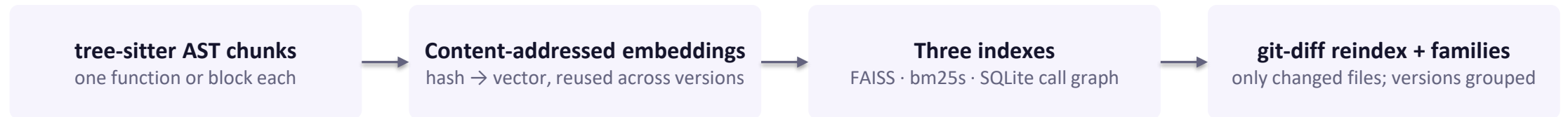
Axiom's answer: fuse all three signals, route by query type, and index versions by content hash.

Our Solutions & Architecture Diagram

ONLINE • PER QUERY



OFFLINE • PER VERSION



The LLM is optional and local. It classifies, expands, decomposes and judges sufficiency. It never reads code; the module boundary enforces that.

Demo & Product Walkthrough

Demo video: youtu.be/QSGzV12PyO4 · Live: axiom-nine-delta.vercel.app

1

axiom index

Builds all three indexes and reports which model rung actually loaded

2

Q1 · semantic

Surfaces preprocessInput in the top 5; the query never names it

3

Q2 · structural

Answers from the call graph, with ordinal evidence for “before”

4

Q3 · usage

An exact deeplink string: BM25 leads and fusion lets it win

5

axiom reindex v1 → v2

Re-embeds only the 50 changed chunks; the other 51 are reused

6

axiom families · no LLM

One function across 3 versions, with diffs; no LLM required

REAL OUTPUT · Q2 ON THE BUNDLED TEST REPO

```
$ axiom query "Which files call
  preprocessInput before resolveTool?"
type STRUCTURAL · 1 pass (sufficient) · 347 ms
```

```
1 src/broken/syntax_error.js:1-11
  dense #2 · sparse #2 · structural #1
  calls preprocessInput (ordinal 0)
  before resolveTool (ordinal 1)
2 src/tools/registry.js:8-15 dispatch
  dense #1 · sparse #1 · structural #2
  calls preprocessInput (ordinal 0)
  before resolveTool (ordinal 1)
```

Run on a bare install with no models downloaded: the hashing fallback embedder, and still the right answer, because the ordering comes from the call graph.

Typers CLI

REST API · 5 endpoints

Streamlit UI

Tools and tech stack used

● Retrieval

- FAISS CPU (flat IP, IVF-PQ at scale)
- bm25s, with a pure-Python fallback
- Weighted Reciprocal Rank Fusion

● Models · CPU, INT8 ONNX

- Qwen3-Embedding-0.6B (primary)
- all-MiniLM-L6-v2 (fallback, measured)
- ms-marco / bge-reranker cross-encoders

● Code structure

- tree-sitter + JavaScript grammar
- SQLite call, import and export graph
- Regex chunker as the fallback rung

● Agent

- Custom bounded loop, no framework
- Heuristic rule engine by default
- Optional local Qwen2.5-1.5B (GGUF); never a third-party API

● Versions

- git diff --name-status, git worktree
- Content-addressed embedding blobs
- Snippet families at cosine ≥ 0.95

● Surfaces & quality

- Python 3.11 · uv · Pydantic v2 contract
- Typer CLI · FastAPI · Streamlit
- MTEB / CoIR eval · pytest, 650+ tests

Impact & Use case

For developers on large JavaScript voice-assistant codebases: dozens of agents and tools, deep directory trees, frequent releases.

● Onboarding

“How is the input normalised before the main handler?” Answered with locations, not guesses.

● Change impact

“Which handlers call `parseIntent` before `resolveTool`?” Answered before a refactor breaks the order.

● Release review

The same function across versions, collapsed into one result with its diffs.

● String & config audits

“Where is the Bluetooth-settings deeplink used?” Every literal usage, exactly.

50 of 101

chunks re-embedded for a 50-file diff; every unchanged chunk is reused

0

embedding calls for renamed, moved or reverted code

100% local

CPU-only. No GPU, and no code ever leaves the machine

Innovation highlights, results and limitations

INNOVATION

- **Order-aware call graph:** answers “X before Y” from source order
- **Per-query-type fusion:** the classifier sets the signal weights
- **Bounded agent:** ≤ 2 passes, 5 s deadline; the LLM never reads code
- **Content-addressed versions:** renames and unchanged files cost nothing

RESULTS · MEASURED

Arm	NDCG@10	MRR@10	Recall@100
BM25 only	0.91	—	—
Dense only	7.59	6.39	27.22
Hybrid RRF	7.78	6.60	27.17

CoIR AppsRetrieval, full test split: 3,765 queries, 8,765 docs. 22M-param MiniLM fallback embedder; reranker not in the scored run.

50 / 101

chunks re-embedded

0

calls for known code

109

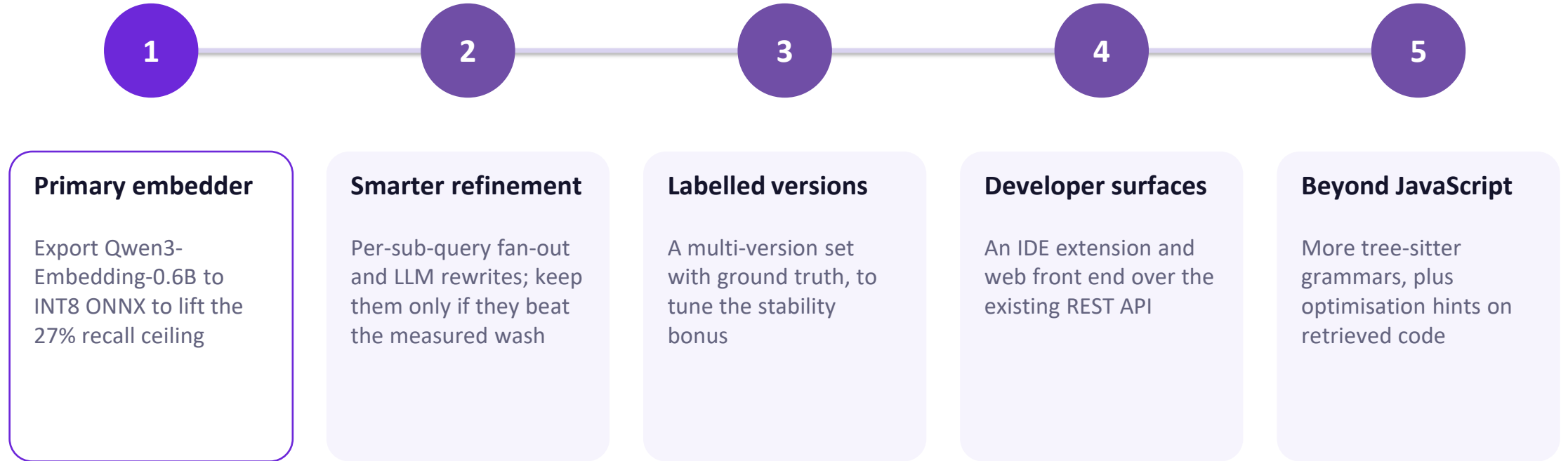
snippet families

LIMITATIONS

- **Recall@100 is 27.2.** The 0.6B primary embedder is not yet exported to ONNX
- **Agent refinement is a wash:** 25 queries helped, 26 hurt out of 592 (Δ NDCG -0.04)
- **Regex fallback chunker** can misattribute calls when tree-sitter is absent
- **JavaScript only**, and no reportable score for the full reranked pipeline yet

Takeaway: recall, not ranking, is the ceiling. About three-quarters of relevant documents never enter the candidate pool, so the lever is a stronger first-stage embedder.

What's next



Rule we keep: every number that reaches a slide is one we ran. Step 1 is first because the measurements say recall is the binding constraint.

Brownie points slide(differentiation)

1 Order-aware call graph

Call lists keep source order, so “X before Y” is answered with ordinal evidence. No embedding of any size can do this.

2 The LLM never reads code

It classifies, expands, decomposes and judges sufficiency, and the module boundary enforces it.

3 Agentic, but bounded

At most 2 passes, with a hard 5 s deadline checked before each pass. It stops as soon as the scores say sufficient.

4 Cannot-fail degradation

No models, no faiss, no bm25s, no tree-sitter: still ranked results, and it names the rung that loaded.

5 Renames cost nothing

Content-addressed blobs: a 50-file diff re-embeds only its 50 changed chunks, and renamed or reverted code costs 0 calls.

6 A harness that refuses itself

Eval will not mark a run reportable while a placeholder is live or the tree is dirty. Corrections stay on the record.

Checklist- Updated on Public GitHub



Working prototype code: public GitHub repo

github.com/HarshdeepAthawale/AXIOM



README with reproducible setup instructions

README.md and docs/Setup.md, `uv sync --frozen`



Demo video, max 5 minutes (YouTube or Drive link)

youtu.be/QSGzV12PyO4



Presentation file (PPT or PDF)

This deck

Thank you

Team Incognito · Axiom: Agentic Code Intelligence

github.com/HarshdeepAthawale/AXIOM

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India